28 | 09 | 26

"Ausencia de efectos secundarios (side effects)":
La ejecución de la función no modifica ningún estado fuera de su ámbito local (por ejemplo, no altera variables globales, no modifica los parámetros de entrada por referencia, ni realiza operaciones de E/S directas que alteran el entorno).

UNIDAD 2

CÁLCULO LAMBDA (λ-CALCULUS)

El cálculo Lambda (λ-calculus), desarrollado en la década de 1930 por el matemático y lógico estadounidense Alonzo Church en la Universidad de Princeton, constituye uno de los pilares fundamentales de la lógica matemática y de la teoría de la computación moderna.

① Orígenes históricos y fundamentos teóricos:

- Contexto de creación: Alonzo Church lo formuló en 1936 como parte de un intento formal para definir la noción general de función y recursividad, antes de la invención de los computadoras modernas.

② Sintaxis y Componentes formales:

El cálculo Lambda puro se construye inductivamente a partir de un conjunto infinito de variables (e.g., x, y, z) y se define mediante la siguiente gramática formal de términos (Λ):

- Variables: Toda variable es un término lambda válido (x).
- Abstracción: Si x es una variable y M es un término, entonces la expresión (λx.M) es una abstracción, la cual representa una función anónima que toma un parámetro x y devuelve el cuerpo M.
- Aplicación: Si M y N son términos, entonces (MN) representa la aplicación de la función M al argumento N.
- Variables libres y ligadas: En la expresión λx.M, la variable x se encuentra ligada dentro del alcance de la abstracción. Si una variable no está ligada a ningún operador λ, se denomina libre.

③ Dinámica de Evaluación: Conversiones y Reducciones

28 | 09 | 26

La computación en este sistema se efectúa mediante transformaciones sintácticas estrictas sobre los términos:

- α-conversión (Alfa-conversión): Permite renombrar variables ligadas para evitar colisiones o la captura indeseada de variables durante la sustitución:

$$\lambda x.M \equiv_\alpha \lambda y.M[x:=y]$$

- β-reducción (Beta-reducción): Es el motor computacional básico. Consiste en aplicar una abstracción a un argumento reemplazando todas las ocurrencias libres de la variable ligada en el cuerpo de la función por el argumento pronto:

$$(\lambda x.M)N \to_\beta M[x:=N]$$

20 | 10 | 26

Elixir y Erlang

Elixir es un lenguaje de programación funcional y moderno que se ejecuta sobre la máquina virtual de Erlang (BEAM).

Tanto Erlang como Elixir son los máximos referentes de la programación funcional aplicada a sistemas distribuidos, concurrentes y tolerantes a fallos.
Aunque operan bajo el paradigma funcional, su verdadero poder radica en cómo combinan este enfoque con el Modelo de Actores para gestionar la concurrencia a nivel masivo.

Es importante aclarar un detalle conceptual: no son lenguajes de "programación lógica" (como prolog), sino lenguajes puramente funcionales y concurrentes. Sin embargo, la herencia de Prolog es sumamente visible en su sintaxis y en herramientas como el Pattern Matching (emparejamiento de patrones)

PILARES DE SU PROGRAMACIÓN FUNCIONAL:
El enfoque funcional en Erlang y Elixir no es solo un capricho académico; es el mecanismo que garantiza que la concurrencia masiva sea segura y libre de errores.
1. Inmutabilidad Absoluta:
En estos lenguajes, las variables no se pueden modificar una vez creadas. Si transformas un dato, se genera una copia modificada en memoria
2. Pattern Matching (Emparejamiento de Patrones)
Heredado directamente de Prolog, no es una simple condicional (if/else), si no una estructura de control de flujo y asignación desestructurada increíblemente potente.
3. Funciones puras y de Primera Clase
las funciones se tratan como cualquier otro tipo de dato (puedes pasarlas como argumentos o retornarlas) y carecen de efectos secundarios ocultos: Una entrada siempre producirá

exactamente la misma salida. Esto facilita enormemente las pruebas unitarias automatizadas.

Comparativa Erlang vs Elixir.
Aunque comparten el mismo motor y capacidades técnicas, sus filosofías de diseño y sintaxis difieren notablemente.

Característica	Erlang	Elixir
Sintaxis	Similar a Prolog; basada en puntuación estricta (., ;, ,). Más académica.	Inspirada en Ruby y Python; limpia, moderna y muy legible.
Metaprogramación	Muy limitada	Soporte nativo de primer nivel a través de Macros (código que escribe código)
Características funcionales clave	Evaluación estricta y asignación única.	Añade el Pipe Operator (|>) para encadenar transformaciones de datos limpiamente.
Uso principal	Infraestructura de redes, switches telefónicos, servidores centrales de telecomunicaciones.	Desarrollo web moderno de alta escala (Phoenix Frameworks), plataformas de mensajería (Discord), e IoT (Nerves)

FUNCIONES DE ORDEN SUPERIOR (MAP, FILTER, FOLD/REDUCE) SOBRE LISTAS.

Las funciones de orden superior (map, filter y fold/reduce) son herramientas clave de la programación funcional que permiten procesar lista de forma declarativa y sin bucles explícitos for o while. Reciben funciones como argumentos para transformar, seleccionar o resumir los elementos de una colección.

1. MAP

Transforma cada elemento de una lista. Aplica una función a cada item y devuelve una nueva lista con el mismo numero de elementos, pero con sus valores modificados.

MAP: Aplica f(x) a cada x de la lista.

Ejemplo: Convertir una lista de números en sus cuadrados.

Python

```
numeros = [1, 2, 3, 4]
cuadrados = list(map(lambda x: x ** 2, numeros))
# Resultado: [1, 4, 9, 16]
```

2. FILTER

Filtra los elementos de una lista según una condición.

Evalúa cada elemento con una función lógica (booleana); si es true, el elemento pasa a la nueva lista; si es false, se descarta.

. Conserva solo los elementos donde f(x) == True.

Ejemplo: Quedarse solo con los números pares.

python

```
numeros = [1,2,3,4,5,6]
pares = list(filter(lambda x: x % 2 == 0, numeros))
# Resultado: [2, 4, 6]
```

3. FOLD / REDUCE

Reduce toda la lista a un único valor. Combina los elementos uno a uno usando una función acumuladora (en muchos lenguajes se llama fold, foldr o reduce).

- Acumula un resultado combinando el acumulador actual con el siguiente elemento.

Ejemplo: Sumar todos los numeros de una lista.

```
python
import functools
numeros = [1, 2, 3, 4]
suma = functools.reduce(lambda acumulador, x: acumlador + x, numeros)
#Resultado. 10 (1+2=3, 3+3=6, 6+4=10)
```

FUNCIÓN	Entrada (ej. lista)	Salida	Propósito principal
Map	[1, 2, 3]	[2, 4, 6] (Mismo tamaño)	Transformar cada elemento.
Filter	[1, 2, 3, 4]	[2, 4] (Menor o igual tamoño.	Seleccionar elementos por condición
FOLD/ REDUCE	[1, 2, 3, 4]	10 (un solo valor)	Combinar o resumir toda la lista.

Característica	PREFIJA	INFIJA	POSTFIJA
Definición	La función se escribe antes de sus argumentos.	El operador se coloca entre los dos operandos.	La función u operador se coloca después de sus argumentos.
Sintaxis general	funcion argumento1 argumento2	argumento1 operador argumento2	argumento1 argumento2 función
Ejemplo matemático	suma 3 5	3 + 5	3 5 suma (notación conceptual)
Ejemplo en Haskell	div 10 2 5	10 + 2 12	10 `div` 2 no es postfija; sigue siendo infija.
Uso de paréntesis	Se utilizan para agrupar expresiones cuando es necesario	Depende de la precedencia y asociativa del operador.	Normalmente requiere una convención o estructura adicional para interpretar la expresión.
Aplicación en Haskell	Es la forma predeterminada para llamar funciones.	Se usa con operadores como +, -, * y funciones entre acentos graves.	No es una forma estándar de aplicación de funciones en Haskell.

03 | 11 | 25

2.3 Intervalos

En Haskell, los intervalos (o ranges) son una notación concisa para generar listas de valores que siguen una secuencia ordenada.

¿Cómo funcionan?

Se escriben utilizando el operador de dos puntos (..) encerrados entre corchetes [...]. Para que un tipo de dato pueda usarse en un intervalo, debe pertenecer a la clase de tipos Enum (como números enteros, caracteres, etc.).

Tipos inter principales de intervalos

1. Intervalo básico (con inicio y fin): Genera una lista desde el primer valor hasta el último (incluyendo ambos, es decir inclusivo).

```
[1..10] -- [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
['a'..'e'] -- ['a', 'b', 'c', 'd', 'e']
```

2. Intervalo con paso (con saltos): Puedes definir los dos primeros elementos para indicarle a Haskell cuál es la diferencia o el "paso" entre cada número.

```
[1,3..10] -- [1, 3, 5, 7, 9] (salto de dos en dos)
[10,8..1] -- [10, 8, 6, 4, 2] (cuenta regresiva)
```

3. Intervalos infinitos (evaluación perezosa / lazy evaluation): Si omites el límite superior, Haskell crea una lista infinita. Gracias a la evaluación perezosa, la computadora no se congela a menos que intentes calcularla por completo; solo genera los elementos que tú le pidas

```
[1..] -- [1, 2, 3, 4, 5, 6, ...] (lista infinita de enteros)
take 5 [1..] -- [1, 2, 3, 4, 5] toma solo los primeros 5.
```

03 | 11 | 26

2.9 OPERADORES

En Haskell los operadores son funciones que se utilizan de forma infija (es decir, colocándose entre sus dos argumentos, como 1+2).

-Características principales

- Son funciones regulares: En Haskell, no hay una distinción real entre una función y un operador, cualquier función binaria puede usarse como operador y viceversa.
- Notación prefija e infija: Haskell utiliza las siguientes reglas sintácticas:
  - De operador (infijo) a función (prefija): Si envuelves un operador entre paréntesis, puede usarse al principio como una función normal.
  - De función (prefija) a operador (infijo): Si se envuelve una función binaria entre comillas simples invertidas (backticks), puedes colocarla entre sus argumentos.
- Principales tipos de operadores en Haskell.

En el Preludio (Prelude) de Haskell vienen definidos por defecto los operadores más comunes:

1. Aritméticos: Suma (+), resta (-), multiplicación (*), división decimal (/), ^ y ^^ (exponenciación, además de funciones enteras como div, mod, rem y quot.
2. Relacionales y de comparación: == (igualdad), /= (desigualdad), <, <=, >, >=.
3. Lógicos (booleanos) && (AND lógico), || (OR lógico)
4. Listas: : (cons/agregar elemento al inicio de una lista), ++ (concatenar listas), !! (obtener el elemento en el índice n)
5. Composición y aplicación de funciones:
   1. . (composición matemática de funciones, ej. f . g).
   2. $ (operador de aplicación que evita el exceso de paréntesis, ej. f $ g x equivale a f (g x)).

2.7 EVALUACIÓN PEREZOSA

La evaluación perezosa (o lazy evaluation) es una estrategia en la que Haskell no calcula el valor de una expresión en el momento en el que se define, sino que retrasa el cálculo hasta que su resultado es estrictamente necesario para continuar el programa.

¿Cómo funciona?

- Bajo demanda: Si defines una variable o una función, Haskell solo crea una promesa (un thunk) de que calculará ese valor si alguien lo pide.
- Sin repetición: Una vez que se calcula por primera vez, el resultado se almacena (memorización) para que no sea necesario volver a procesarlo si se vuelve a usar.
- Comportamiento por defecto: En Haskell, prácticamente todas las expresiones son perezosas de manera predeterminada, a diferencia de lenguajes de evaluación estricta (voraz) como C++, Java o Python (por defecto)

VENTAJAS CLAVES EN EL DESARROLLO

1. Estructuras de datos infinitas: Puedes definir listas sin fin porque Haskell solo calculará los elementos que realmente le pidas.
2. Mayor eficiencia computacional: Evita procesar argumentos o ramas de código que nunca se utilizan en el resultado final.
3. Modularidad: Permite separar la definición de los datos (qué son) del flujo de control (cuántas necesito).